

12/09/2020

NLP + WORD EMBEDDINGS + Seq2Seq + ATTENTION + TRANSFORMERS + BERT + GPT +

1. NLP - Natural language processing -

A field of AI that enables computers to understand, process and generate human language.

→ Common applications:

- Sentiment analysis

- Spam detection

- Translation

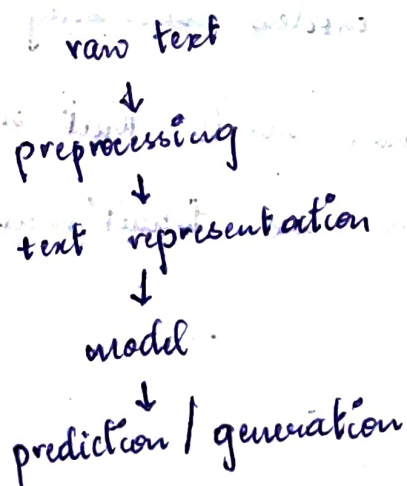
- Chatbots

- Text classifications

- Question answering

- Summarization

→ Basic flow -



2. Text Preprocessing - Cleaning and transforming raw text before giving it to an NLP model.

→ Common steps:

- Lowercasing

- Removing punctuation

- removing unnecessary symbols

- tokenization

- stopword removal

- stemming

- lemmatization

example: "The cats are running!"

↓

"the cats are running" — lowercase

↓ tokenize

["the", "cats", "are", "running"]

3. Tokenization - Splitting text into smaller units called tokens.

Word tokenization: "I love AI" → ["I", "love", "AI"]

Subword: "unhappiness" → un + happy + ness

* Modern transformers models commonly use subword tokens.

4. Stopwords - Very common words that may carry little useful information for some traditional NLP tasks.

examples: the

is

a

an

and

of

* Modern transformer models usually do not require stopword removal.

5. Stemming - Reduces words to a crude root form by removing suffixes.

example: i: playing

played

plays → play

ii: studies → study

* It is fast but may produce non-dictionary words.

6. Lemmatization - Converts a word to its proper dictionary

base form.

example:

running → run

better → good

mouse → mouse

Comparing with stemming:

Stemming → rough root

Lemmatization → meaningful dictionary form

7. N-Grams - A sequence of N consecutive tokens.

example sentence:

"I love machine learning"

Unigram ↓

I
love
machine
learning

Bigram ↓

I love
love machine
machine learning

Trigram ↓

I love machine
love machine learning

* N-grams help capture some local word order.

8. Bags of Words - BoW - A text representation that converts documents into vectors using word occurrence

Counts.

example documents :

D₁ : I love AI

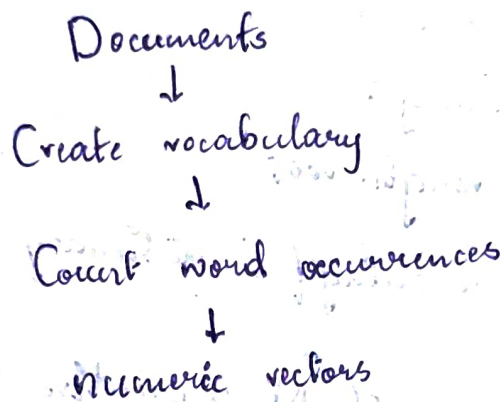
D₂ : I love ML

Vocabulary: [I, love, AI, ML]

Vectors: D₁ → [1, 1, 1, 0]

D₂ → [1, 1, 0, 1]

Working :



Advantages :

- Simple
- Easy to understand
- Works for basic text classification

Limitations :

- Ignores word order
- " semantic meaning
- Produces sparse vectors
- Vocabulary can become huge.

9. TF-IDF - A text representation that gives higher importance to words that are frequent in one document but uncommon across all documents.

Term Frequency - Inverse Document Frequency

Term Frequency $TF(t, d) = \frac{\text{No. of times the term appears in the document}}{\text{Total number of words in the document}}$

Inverse Document Frequency = $\log \left(\frac{\text{Total no. of documents}}{\text{No. of documents containing the term}} \right)$

$$TF-IDF(t, d) = TF(t, d) \times IDF(t)$$

BoW

1. Counts words
2. Treats all occurrences similarly
3. Simple
4. Sparse

TF-IDF

1. Weights words
2. Reduces wt. of very common words
3. More informative
4. Sparse

WORD EMBEDDINGS

10. Word Embeddings - Dense numerical vectors where semantically similar words have similar vector representations.

example:

king $\rightarrow [0.2, 0.8, 0.4, \dots]$

Queen $\rightarrow [0.3, 0.7, 0.5, \dots]$

Car $\rightarrow [0.9, 0.1, 0.2, \dots]$

Unlike BoW / TF-IDF $\rightarrow$ Sparse

Embeddings $\rightarrow$ dense

Embeddings capture semantic relationships.

14/09/2026

11. Cosine Similarity - Measures similarity between two vectors based on the angle between them

$$\cos \theta = \frac{A \cdot B}{\|A\| \|B\|}$$

Interpretation:

Near 1 $\rightarrow$ very similar

Near 0 $\rightarrow$ unrelated

Near -1 $\rightarrow$ opposite direction

$\rightarrow$ Word2Vec - A neural network based technique that learns word embeddings from surrounding word context.

Main architectures:

- CBOW
- Skip-Grass

1. CBOW - Continuous Bag of words : Predicts a target word using surrounding context words.

example : " The cat _____ on the mat "

Context : The , cat , on , the , mat

Predict : sat

Advantages:

- Faster
- Works well with frequent words.

2. Skip-Grass - Predicts surrounding context words using a target word.

example : Target : cat

Predict nearby words: the
sat
on

Usually works better for rare words than CBOW

CBOW

1. Context $\rightarrow$ target word.
2. Faster
3. Good for frequent words.
4. Efficient on large datasets

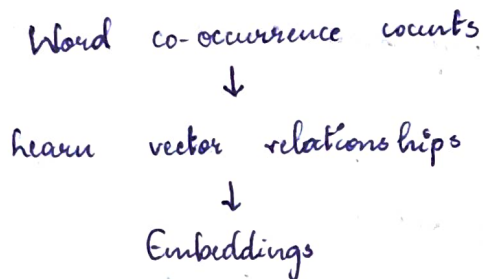
Skip-Gram

1. Target word $\rightarrow$ context
2. Slower
3. Better for rare words
4. Can learn richer representations

$\Rightarrow$ GloVe - A word embedding technique that learns vector representations using global word co-occurrence statistics.

GloVe - Global Vectors for Word Representations

Main idea:



Difference:

WordVec $\rightarrow$ learns from local context windows.

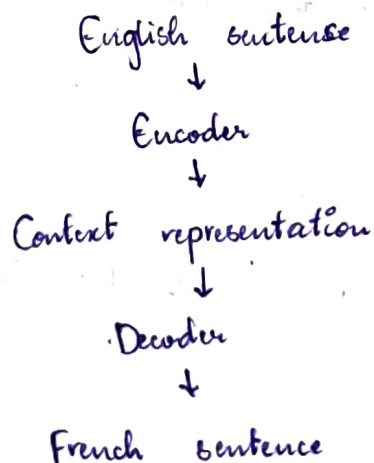
GloVe $\rightarrow$ uses global co-occurrence statistics.

SEQ2SEQ

12. Sequence-to-Sequences - A neural network architecture that converts one sequence into another sequence.

Common use:

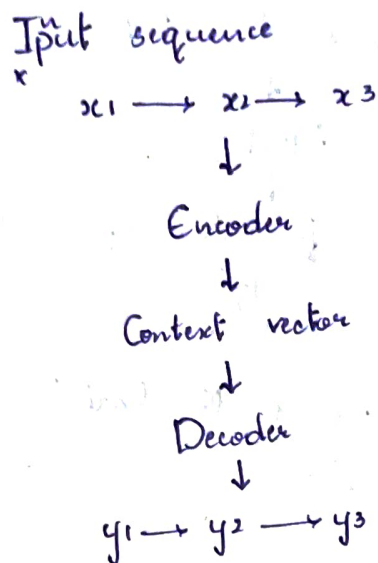
Translation, Summarization, Speech Recognition.



→ Encoder - Decoder Architecture -

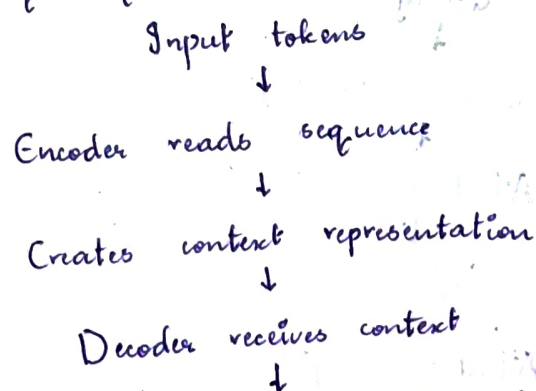
Encoder : Processes the input sequence and creates an internal representation.

Decoder : Uses that representation to generate the output sequence.



Traditional Seq2Seq often used : 1. RNN 2. LSTM 3. GRU

Working of Seq2Seq :



Generates output one token at a time

→ Teacher Forcing : During training, the decoder is given the correct previous output instead of always using its own previous prediction.

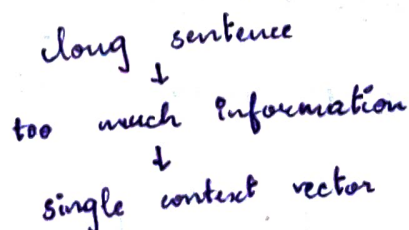
Advantage : Faster and more stable training.

Limitation : Training and inference conditions differ.

→ Seq2Seq Limitation -

Traditional Encoder-Decoder compresses the whole input into one fixed-size context vector

Problem :

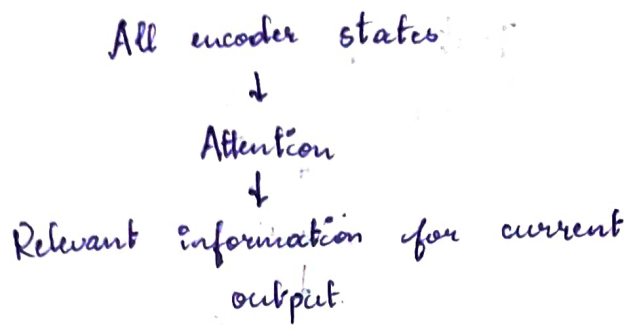


This bottleneck led to Attention.

ATTENTION

13. Attention Mechanism - A mechanism that lets a model focus on the model relevant parts of the input while producing each output.

Instead of relying on one fixed context vector:



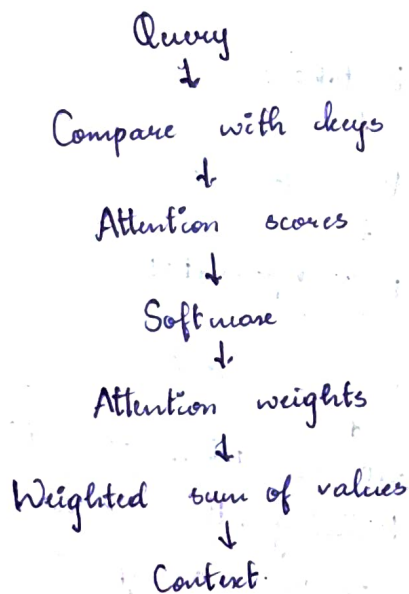
example:

Input: "The dog chased the ball"

while generating: "chien"

model focuses strongly on: "dog"

Working:



* Terms:

- Query (Q) - What information am I looking for?
- Key (K) - " " does each token contain?
- Value (V) - The actual information associated with that token

→ Attention Score -

$$\text{score}(Q, K) = QK^T$$

then apply softmax:

$$\text{Attention Weights} = \text{softmax}(QK^T)$$

final output:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T)V$$

→ Scaled Dot-Product Attention:

Transformers use:

$$\text{Attention}(Q, k, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

where,

Q = query

V = value

k = key

d_k = dimension of keys

* Why divide by $\sqrt{d_k}$?

To keep very large dot-product values from making softmax unstable.

→ Softmax - converts scores into probabilities that sum to 1.

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

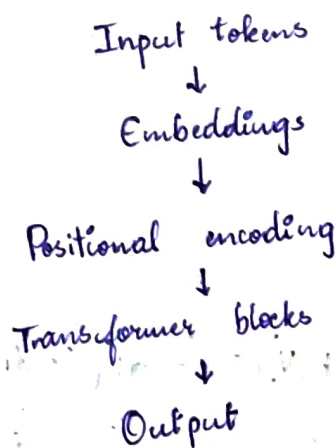
scores: 2, 1, 0 → prob: 0.67, 0.24, 0.09

TRANSFORMERS

14. Transformer - A neural-network architecture that processes sequences using attention mechanisms rather than recurrent connections.

Transformer → attention-based parallel processing

Architecture -



Original Transformer:

Encoder + stack

Decoder stack

→ Self Attention - Attention where Query, key and value all come from the same sequence.

example: "The animal didn't cross the street because it was tired."

↓
self-attention helps determine that "it" refers to "animal"

Self Attention Working -

Input embeddings : X

Generate :

$$Q = XW_Q$$

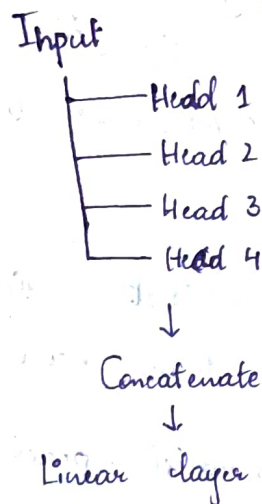
$$K = XW_K$$

$$V = XW_V$$

Then:

$$*** \text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

→ Multi-Head Attention - Runs several attention operations in parallel so different relationships can be learned simultaneously.



Different heads may learn:

- Syntax
- long-distance relationships
- entity references
- semantic relationships

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

→ Positional Encoding - Transformers process tokens in parallel, so they need information about token order.
→ Adds position information to token embeddings.

Word Embedding + Position Encoding

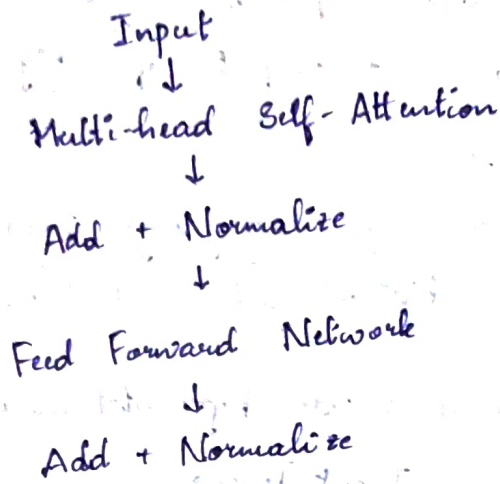
↓
Transformer

Classic sinusoidal formulas:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

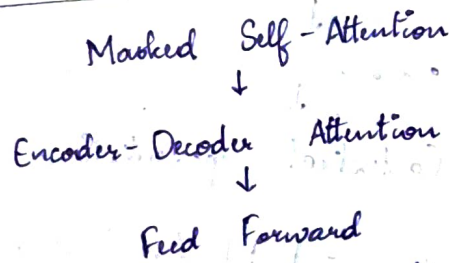
→ Transformer Encoder Block -



Important components:

- Self-attention
- Feed-forward network
- Residual connections
- Layer normalization

→ Transformer Decoder Block -



* Masked attention prevents the decoder from looking at future tokens while generating.

→ Masked Self-Attention - Prevents a language model from seeing future tokens.

example: " I love deep "

Predicts: " learning "

While predicting "learning", it cannot use tokens that come after it.

Transformer Advantages -

- Parallel training

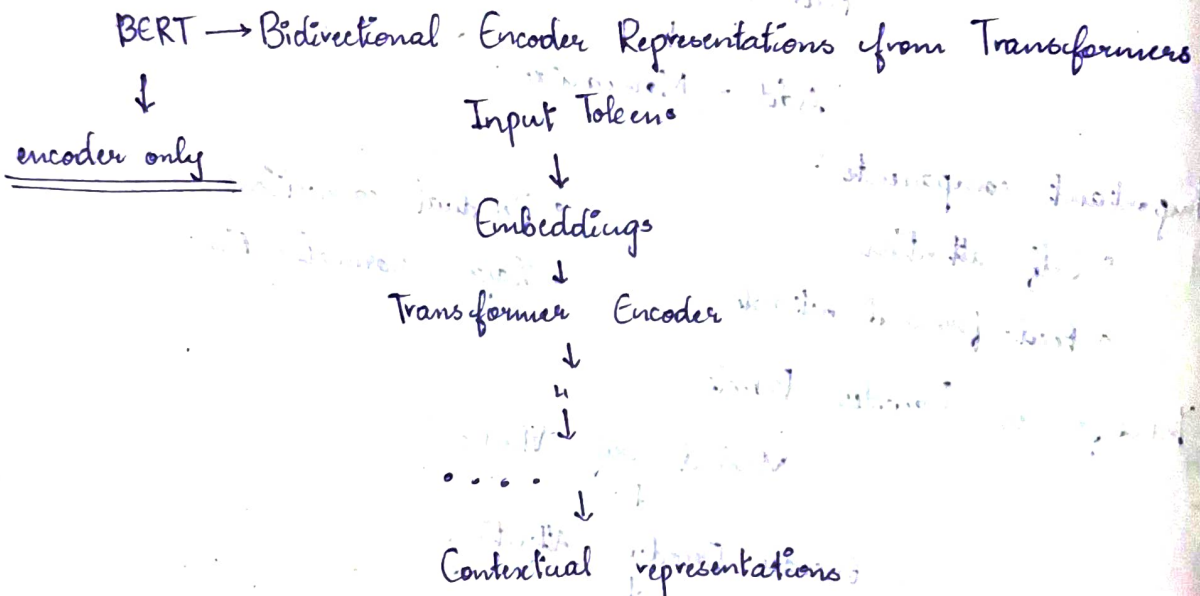
- Handles long-range dependencies better than basic RNNs.
- Scales well
- Foundation of modern LLMs

Limitations :

- High memory usage
- Attention cost grows with sequence length
- Needs large data and compute for big models.

BERT

15. BERT - A transformer-based language model that uses encoder blocks and learns bidirectional contextual representations.



→ Bidirectional Understanding - BERT looks at both left and right context.

example: "I went to the bank to deposit money".

and, "I sat on the bank of the river".

* BERT uses surrounding context to understand the meaning of "bank".

→ Masked Language Modeling - MLM

Some input words are hidden and the model predicts them.

example: "The cat sat on the [MASK]".

↓

"mat"

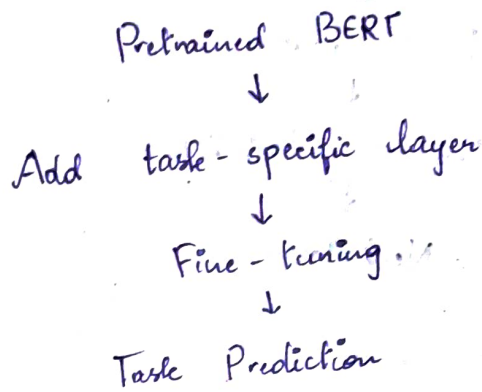
* This helps BERT learn context.

→ BERT Fine-Tuning

A pretrained BERT model can be adapted to tasks such as:

- Sentiment analysis
- Text classification
- Named entity recognition
- Question answering

Flow -



GPT

16. Generative Pre-trained Transformer - A transformer-based language model that uses decoder blocks and generates text autoregressively by predicting the next token

GPT → decoder only

→ Autoregressive generation - Generates one token at a time using previously generated tokens.

example: Input: "Machine learning is"

Predict: "a"

"Machine learning is a"

Predict:

"field"

→ Next-Token Prediction -

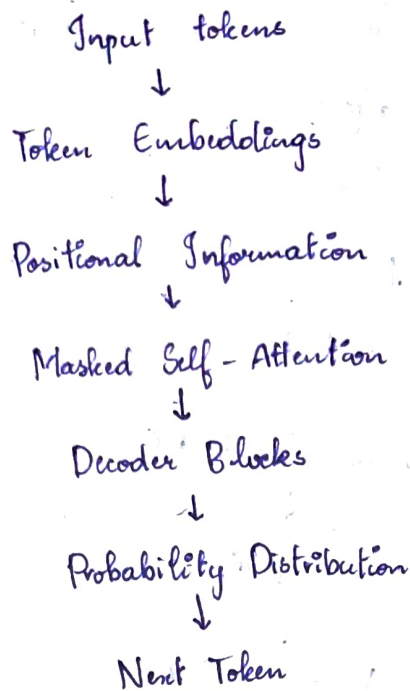
$$P(x_t | x_1, x_2, \dots, x_{t-1})$$

meaning: Probability of next token given all previous tokens.

example: "I love machine"

Target: "learning"

→ GPT Working -



Repeat until generation ends.

BERT

1. Encoder-only
2. Bidirectional
3. Understands context
4. Masked language modeling
5. Strong for classification/QA.

GPT

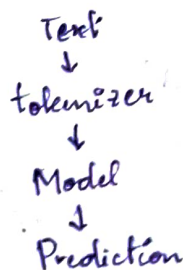
1. Decoder-only
2. Autoregressive
3. Generates text
4. Next-token prediction
5. Strong for generation.

HUGGING FACE

17. Hugging face - An ecosystem and Python library that provides pretrained Transformer models, tokenizers, datasets and tools.

Main library: transformers

Workflow -



→ Tokenizer - Converts raw text into token IDs that a transformer model can process.
example:
["Hello", "World"]
↓
[7592, 2088]

→ Pretrained Model

Hugging Face allows loading pretrained models instead of training from scratch.

Concept - from transformers import AutoTokenizer, AutoModel

→ Pipeline - A simple Hugging Face interface for performing NLP tasks with pretrained models.

Concept - from transformers import pipeline

example - classifier = pipeline ("sentiment-analysis")

then - classifier ("I love this movie")

FINE-TUNING CONCEPT

Fine tuning - Continuing training of a pretrained model on a smaller task-specific dataset.

Large - Pretrained model

↓

Task Dataset

↓

Fine tuning

↓

Specialized Model

Why useful?

↳ scratch → expensive

↳ fine-tuning → cheaper + faster

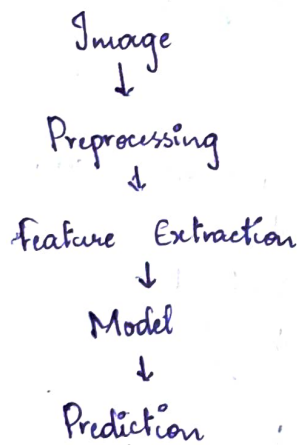
COMPUTER VISION + TRANSFER LEARNING + OBJECT DETECTION + SEGMENTATION + ViT

1. Computer Vision - A field of AI that enables computers to understand and analyse images and videos.

Common tasks:

- Image Classification
- Object Detection
- Image Segmentation
- Face recognition
- Pose estimation
- OCR

Flow -



→ Image Representation - A digital image is a matrix of pixel values.

Grayscale : $h \times w$

RGB : $h \times w \times 3$

Typical pixel range : 0 to 255

often normalized : $x' = \frac{x}{255}$

so value becomes : 0 to 1

OpenCV : An open-source computer vision library used for image and video processing.

Import : `import cv2`

Common operations: read image, resize, crop, convert color, blur.

edge detection, thresholding, contours, video processing.

⇒ Reading and Displaying an Image

→ `image = cv2.imread("image.jpg")`

OpenCV reads images in: BGR not RGB

Convert:

→ `rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)`

⇒ Resize

→ `resized = cv2.resize(image, (224, 224))`

Used because neural networks usually expect fixed input dimensions.

⇒ Grayscale Conversion

→ `gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)`

Useful for:

- Edge detection
- Thresholding
- Classical CV

⇒ Gaussian Blur - Smooths an image and reduces noise

→ `blur = cv2.GaussianBlur(image, (5, 5), 0)`

⇒ Edge Detection - Finds strong changes in pixel intensity.

common method: Canny Edge Detection

→ `edges = cv2.Canny(gray, 100, 200)`

⇒ Thresholding - Converts image pixels into groups based on an intensity threshold.

example:

→ `binary = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)`

basic idea:

$$\text{output} = \begin{cases} 255 & x > T \\ 0 & x \leq T \end{cases}$$

⇒ Contours - Curves joining points along object boundaries.

→ contours, _ = cv2.findContours(
binary,
cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE
)

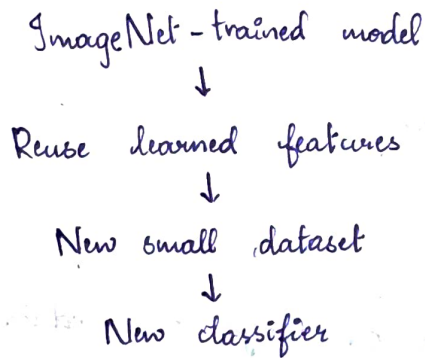
Useful for:

- Shape detection
- Object boundaries
- Measurement

TRANSFER LEARNING

2. Transfer Learning - Reusing a model trained on a large dataset for a new but related task.

example:



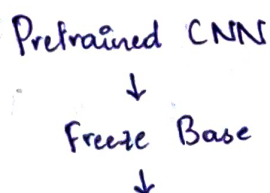
Why useful:

- less training data
- better performance
- faster training
- less compute

→ Feature Extraction - In transfer learning, early CNN layers often learn general features:

- Edges
- Textures
- Shapes

So we can freeze them.



Add New Classifier



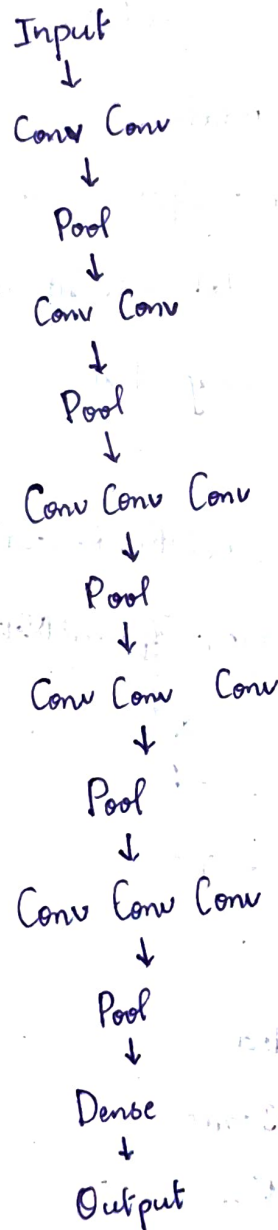
Train New layers

→ Fine-tuning Concept -

Unfreezing some pretrained layers and training them slightly on the new dataset.

3. VGG: A CNN architecture known for repeatedly using 3×3 convolutions.

VGG-16:



Advantages -

- Simple architecture
- Easy to understand

Limitations -

- Very large
- Many parameters
- Slow

4. ResNet

5. MobileNet - A lightweight CNN designed for mobile and edge devices

Main idea: Depthwise Separable Convolution

Instead of normal convolution: Standard Conv

MobileNet uses: $\text{Depthwise Conv} + \text{Pointwise } 1 \times 1 \text{ Conv}$

This reduces $\rightarrow$ $\circ$ parameters $\circ$ computation $\circ$ model size

$\rightarrow$ Depthwise Separable Convolution

Depthwise Convolution - Applies one filter per input channel

Pointwise Convolution - Uses 1×1 convolutions to combine channels

6. EfficientNet - A CNN family that scales network depth, width and image resolution together.

3 dimensions: Depth, Width, Resolution

Instead of increasing only one, EfficientNet uses: Compound Scaling

$$\text{depth} = \alpha^\Phi$$

$$\text{width} = \beta^\Phi$$

$$\text{resolution} = \gamma^\Phi$$

where Φ controls model scaling

Comparison

<u>Model</u>	<u>Main idea</u>	<u>Best Use</u>
VGG	Stacked 3×3 convs	Simplicity
ResNet	Skip connections	Deep Networks
MobileNet	lightweight convs	Mobile/edge
EfficientNet	Compound Scaling	Accuracy + efficiency

OBJECT DETECTION

7. Object Detection - Identifies both the class and location of objects in an image.

Output :

Class label + Bounding box

Example :

Image



Person → box

Car → box

Dog → box

→ Bounding Box - Usually represented as: $(x_{min}, y_{min}, x_{max}, y_{max})$
(or)
 $(x_{center}, y_{center}, width, height)$

→ IoU - Intersection over Union - Measures overlap between predicted and ground-truth bounding boxes.

$$IoU = \frac{\text{Area of Intersection}}{\text{Area of Union}}$$

$IoU = 1$ → perfect overlap

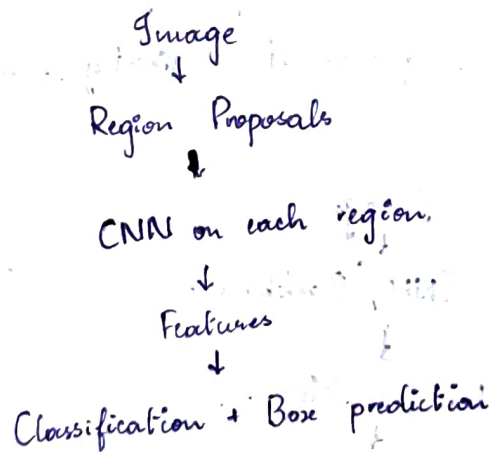
$IoU = 0$ → no overlap

Common threshold : $IoU \geq 0.5$ maybe treated as good detection.

R-CNN FAMILY

8. R-CNN - An object detection model that first proposes candidate regions, then runs a CNN on each region.

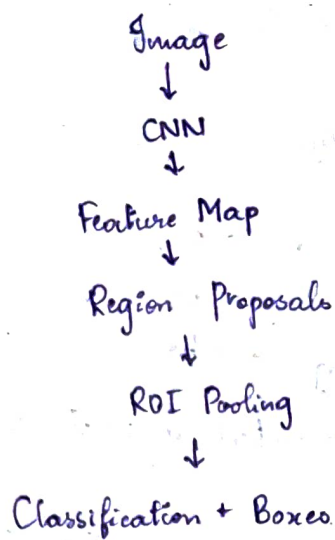
Working -



Problem : Very slow

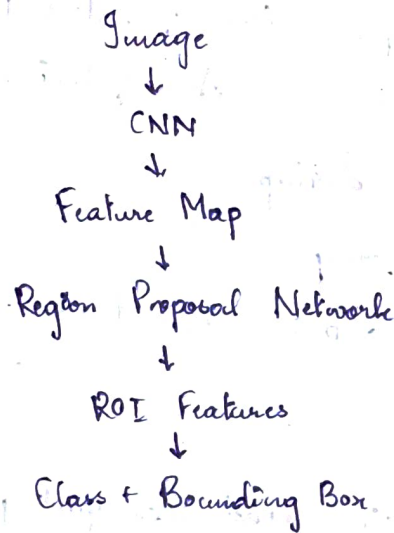
15/09/2026

→ Fast R-CNN - Runs the CNN once on the entire image and extracts features for proposed regions from the shared feature map.
* Faster than CNN



→ Faster R-CNN - Replaces external region proposal methods with a Region Proposal Network.

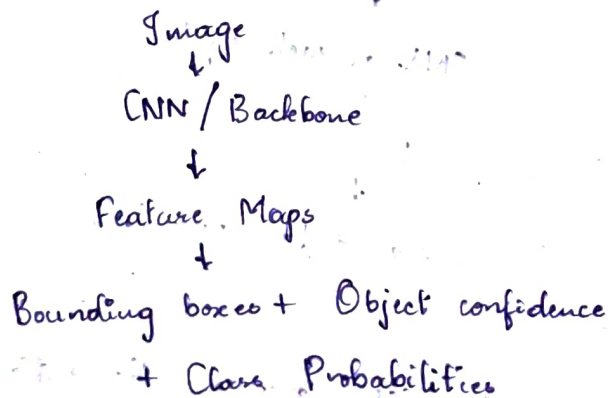
Main idea: RPN → learns region proposals



YOLO

9. YOLO (You only look once) - A one-stage object detector that predicts bounding boxes and class probabilities directly from an image in a single forward pass.

Flow:



→ Two-Stage Vs One-Stage Detection

Two-Stage examples: Faster R-CNN

First Find regions then classify regions

One Stage example: YOLO

- directly predicts: Boxes + Classes

Two - Stage

1. Usually high accuracy
2. Region proposals first
3. Faster R-CNN

One - Stage

1. Usually faster
2. Direct prediction
3. YOLO

→ YOLO Prediction Concept -

For each detection, the model predicts something like:

x
y
width
height
confidence
class probabilities

→ Non - Maximum Suppression - NMS - Removes duplicate bounding box

that represent the same object.

Example:

Person
↓
Box A confidence: 0.95
" B " 0.88
" C " 0.60

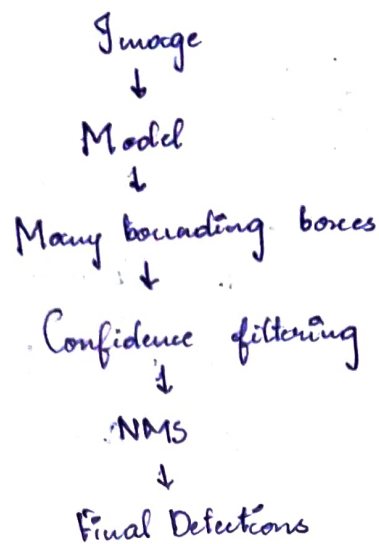
If boxes heavily overlap:

keep highest confidence, remove overlapping duplicates

Working -

Sort boxes by confidence
↓
Keep highest - confidence box
↓
Compare IoU
↓
Remove boxes with high overlap
↓
Repeat

→ Detection Pipeline -



SEGMENTATION

10. Image Segmentation - Assigns a class label to pixels in an image.

Instead of: one bounding box

We predict: pixel-level regions

→ Semantic Segmentation - Assigns every pixel to a class but does not distinguish individual objects of the same class.

example: All cars → same class

→ Instance Segmentation - Identifies each separate object instance at the pixel level.

Example:

Car 1

Car 2

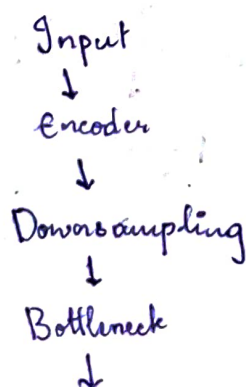
Car 3

each gets its own mask

U-NET

11. U-NET - A CNN architecture for image segmentation with an encoder-decoder structure and skip connections.

Architecture:



Upsampling
↓
Decoder
↓
Segmentation mask

But importantly:

Encoder layer → Decoder layer
skip connection

Shape:



hence the name U-Net.

→ U-NET Encoder -

Conv
↓
ReLU
↓
Conv
↓
Pooling

Extract features
Reduce spatial dimensions

→ U-NET Decoder -

Upsampling
↓
Combine skip features
↓
Convolution
↓
Segmentation Output

Purpose: Restore spatial resolution

→ U-NET Skip Connections - Skip connections copy high-resolution encoder features to decoder layers.

Why? Because deep encoder layers lose spatial detail.

Skip connections restore -

- Boundaries
- Fine details
- Location information

→ Segmentation Output -

$h \times w \times \text{classes}$

Each pixel receives probabilities

For binary segmentation, sigmoid is commonly used.

For multiclass segmentation, softmax is common.

VISION TRANSFORMER

12. Vision Transformer - ViT - A model that applies Transformer architecture to images by dividing them into patches and treating patches like tokens.

ViT → image + patches

→ Image Patches -

example: 224×224 image

Patch size: 16×16

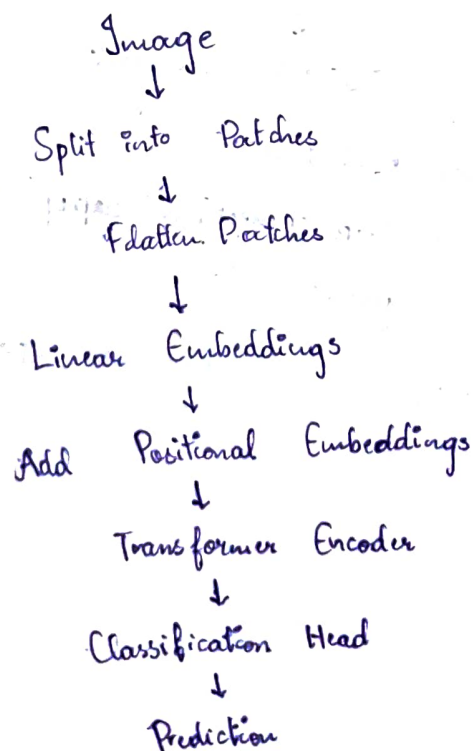
Number of patches : $N = \frac{H \times W}{p^2}$

For : 224×224

with : 16×16

patches : $N = 196$

Architecture -



→ Patch Embedding - Each image patch is flattened and projected into an embedding vector.

$$z_i = x_i W_E$$

where x_i = flattened image patch

W_E = learned projection

→ CLS Token -

ViT often adds a special: [CLS] token.

[CLS] + Patch 1 + Patch 2 +

After transformer processing, the CLS token representation is used for classification.

→ ViT Self-Attention -

Just like NLP Transformers:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

learns btw image patches relationships.

CNN

1. Uses convolution
2. Strong local bias
3. Good with smaller datasets
4. Pixels → filters.

ViT

1. Uses self-attention
2. Learns global relationships.
3. Often benefits from larger datasets.
4. Patches → tokens